SVKM'S NMIMS® Deemed to be UNIVERSITY | INDORE

Image and Video Processing: Mini-Project Report

J.A.R.V.I.S. HUD

A Touchless, Voice-Controlled Real-Time Vision Assistant

(inspired by the Iron Man heads-up display)

Course: Image and Video Processing

Programme / Semester / Division: MBA Tech/ Semester V

Student name	Roll number
Gurmann Singh Chiraunde	H037
Disha Singh Kushwaha	H001
Palash Jain	H024

Faculty: Dr. Raj Gaurav Mishra

Academic Year: 2026-27

Date of submission: 13 October 2026

Abstract

Image-processing software is normally controlled with a keyboard and mouse. This is difficult when the user's hands are busy, gloved or must stay clean, and for people with limited hand movement. This project builds a touchless vision assistant that runs on an ordinary laptop webcam and is controlled only by voice. A real-time heads-up display (HUD) finds, tracks and recognises faces, shows their distance and direction, and offers night-vision, thermal and motion-alert modes. Spoken commands such as “Hey Jarvis, find the edges” apply an image-processing operation to the live frame, and a synthesised voice replies using the recognised person's name. The system uses techniques from all eight units of the course, including point processing, histogram equalisation, spatial and frequency-domain filtering, the DFT and DCT, morphology, Sobel and Canny edge detection, segmentation, colour models, and I/P-frame motion estimation, built with Python and OpenCV. Running the face detector only on every fifth frame keeps the system at 35 frames per second with four people in view. Voice commands were understood in 592 of 592 tests, and face recognition gave correct decisions in 22 of 23 tests.

1. Introduction

1.1 Background

Almost every image-processing tool assumes that the user sits at a desk with a keyboard and mouse. In many real situations this is not true. A technician repairing a circuit board holds tools in both hands, a doctor or a food handler should not touch shared devices, a lab worker wears gloves, and people with hand injuries or tremor find precise mouse work slow or painful. At the same time, every laptop now has a webcam, and voice assistants such as Siri and Alexa are common. Voice assistants, however, only answer questions; they do not process what a camera sees. A heads-up display (HUD), as used in fighter planes, car windscreens and smart glasses, shows information directly on top of the live view so the user never has to look away.

1.2 Why the problem is relevant

Touchless interaction has become more important since the COVID-19 pandemic, and accessible software should be usable without a mouse. A system that lets a person point a camera at something and say “sharpen this” or “find the edges”, and see the result at once, meets both needs. Working offline on a normal laptop also keeps camera and voice data private and needs no internet connection or special hardware.

1.3 Role of image and video processing

Every part of the assistant is an image- or video-processing task. In poor light the frame must be enhanced (point processing, histogram equalisation, filtering). To answer analysis commands, edges, regions and frequency content must be extracted. Colour models must be chosen so that segmentation still works when the lighting changes. Because the input is a live video at 30 frames per second, video processing is also needed. Consecutive frames are almost the same (temporal correlation), so the system borrows the I-frame and P-frame idea from video compression: it fully detects faces on an I-frame and only estimates their motion on the P-frames in between.

1.4 Motivation

The idea came from J.A.R.V.I.S., the voice assistant and helmet display in the *Iron Man* films: one interface that sees what the user sees, adds useful information on top, and responds to speech. Building a working version was a good way to bring every unit of the syllabus together in one live system instead of separate programs, and it made us measure things that only matter in real time, such as speed, frame rate and tracking accuracy.

2. Problem Statement and Objectives

2.1 Problem statement

- Image-processing tools need keyboard and mouse input, which is impractical when the hands are busy, gloved or must stay clean, and is a barrier for people with limited hand movement.
- Voice assistants do not process a live camera view, and normal image-processing tools are not voice-controlled or real-time on ordinary laptops.
- **Problem addressed:** build a low-cost, offline, hands-free system that lets a user view and process a live camera feed in real time using only spoken commands, with results shown on the view and spoken feedback.

Overall objective: The objective of this project is to build a voice-controlled, real-time heads-up display that detects, tracks and recognises people in a webcam stream and applies image- and video-processing operations to the live view on spoken command, using techniques from all eight units of the course.

2.2 Specific objectives

1. **Real-time HUD.** Detect, track and recognise faces in a 640×480 webcam stream at 30 frames per second or more, showing distance, direction and name, and use motion estimation so the detector does not have to run on every frame.
2. **Hands-free control.** Provide offline voice control with a wake word (“Hey Jarvis”), flexible wording and spoken replies, with at least 95 % of commands understood and no wrong commands from unrelated speech.
3. **Image processing on demand.** Implement operations from every syllabus unit (enhancement, transforms, morphology, edges, segmentation and colour), each applied to the live frame in under one second and shown with its intermediate steps.
4. **Viewing modes.** Provide night vision for low light, thermal pseudo-colour, and a motion alert based on frame differencing.
5. **Measured results.** Measure speed, frame rate, tracking accuracy, compression, recognition accuracy and voice-command accuracy instead of judging the output only by eye.

3. Scope of the Project

3.1 What the project covers

- Live capture from a normal laptop webcam and processing of every frame on the CPU.
- Face detection, tracking between detections, face recognition with on-the-spot registration (“remember me”), and distance and direction estimation.
- A HUD in the style of the Iron Man helmet: targeting reticle, scales, radar, status panel and name tags.
- Four live modes (night vision, thermal, radar, motion alert) and twelve one-shot operations: x-ray (digital negative), sketch, cartoon, Canny edges, Sobel gradient, DFT spectrum, low-pass smoothing, high-boost sharpening, background removal, skin segmentation, thermal and histogram equalisation. Every snapshot also reports its PNG and JPEG file sizes.
- Offline voice control for 24 commands, accepting many ways of saying each one.
- Saving every result, a keyboard shortcut for every command, 82 automated tests, and a script that regenerates every result in this report.

3.2 What the project does not cover

- **No medical or scientific measurement.** “X-ray” is a digital negative and “thermal” is a pseudo-colour version of brightness, not temperature; no infrared or depth camera is used.
- **Not a security system.** Face recognition is meant for a few registered users; it was not tested on large face databases or against fake faces such as printed photos.
- **No video file compression.** Compression is studied on single frames (JPEG and PNG). The I/P-frame idea is used for tracking, not to encode video files, and B-frames are not used.
- **No general conversation.** Only English commands from a fixed (but flexible) list are understood.
- No graphics-card acceleration, mobile version or multiple cameras.

3.3 Type of images and video

Colour video (three 8-bit channels per pixel) from the built-in webcam at 640×480 pixels and 30 frames per second, taken indoors under normal room or classroom lighting, with one or more people facing the camera. Audio is recorded from the laptop microphone at 16 kHz.

3.4 Application domain

- **Assistive computing:** users who cannot comfortably use a mouse or keyboard.
- **Hands-busy inspection:** labs, workshops and repair benches, where the operator asks for edges, sharpening or enhancement without touching the computer.
- **Education:** a live demonstration of image-processing concepts on the student's own camera image.
- **Simple monitoring:** recognition and the motion alert are the basis of a presence, attendance or intrusion monitor.

4. Dataset / Input Description

The system works on a live camera stream, so there is no fixed external dataset. For development and for the results in this report we used the inputs in Table 1. All the figures in this report use one reference image: a photograph of our project team, resized to the webcam's 640×480 resolution (Figure 1). It has four people at different distances and a bright, detailed background, which makes it a harder test than a single face.

Table 1. *Inputs used for development and evaluation*

Input	Source	Count / size	Resolution and format
Live webcam stream	Laptop built-in camera	Continuous, in every live session	640×480, 30 frames per second, RGB 8-bit
Reference image (team photo)	Smartphone photograph of the team; resized from 1280×960 to 640×480 (the webcam size)	1 image, 210 KB as WebP	1280×960 WebP, used at 640×480
Registration frame	One webcam frame of Gurmann from an earlier session, used only to register his face for recognition	1 frame	640×480, PNG
Synthetic test image	Made by a script: gradients, shapes, 16-pixel stripes (a known DFT peak), text and noise	1 image	640×480, JPEG
Synthetic video	The team photo moved along a known path (up to 70 pixels sideways, 25 up and down), with slight flicker and noise, so the true face position is known in every frame	150 frames (5 s at 30 fps)	640×480
Live-session results	Results saved automatically while using the system on 2 days	56 images	640×480, PNG / JPEG
Speech test set	Every command phrase after “hey jarvis”, spoken by 4 computer voices, plus 10 unrelated sentences per voice	592 commands + 40 unrelated	16 kHz, mono, WAV
Pre-trained models	Vosk small English (speech), YuNet (face detection), SFace (face recognition), Haar cascade (classical face detector)	41 MB, 0.2 MB, 39 MB, 0.9 MB	Model files

The 56 live-session results break down by command as xray 13, edges 12, snapshot 7, cartoon 5, remove_bg 5, lowpass 4, spectrum 3, highpass 2, skin 2, sobel 2, enhance 1. As in the notes, a digital image is a function f(x,y), and a colour image is three matrices R, G and B. Figure 2 shows the reference image split into these matrices. With a spatial resolution of 640×480 and a gray-level resolution of 8 bits (L = 256 levels, 0 to 255), one frame holds 640 × 480 × 3 = 921,600 bytes.

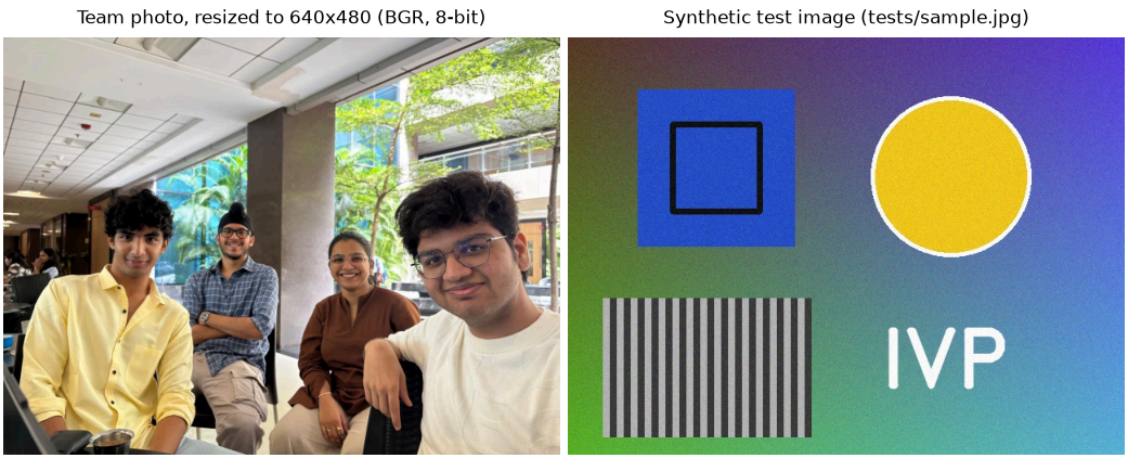


Figure 1. *Inputs: the team photo used for every figure in this report (left) and the synthetic test image used by the automated tests (right).*



Figure 2. *The reference image as matrices: R, G and B channels, grayscale, and the hue and saturation channels of the HSV colour model.*

5. Methodology / Proposed Approach

The system has two parts that run at the same time and meet at a command queue (Figure 3). Every processing step follows one rule: it takes a 640×480 colour frame and returns a frame of the same size, so each operation can be built and tested on its own.

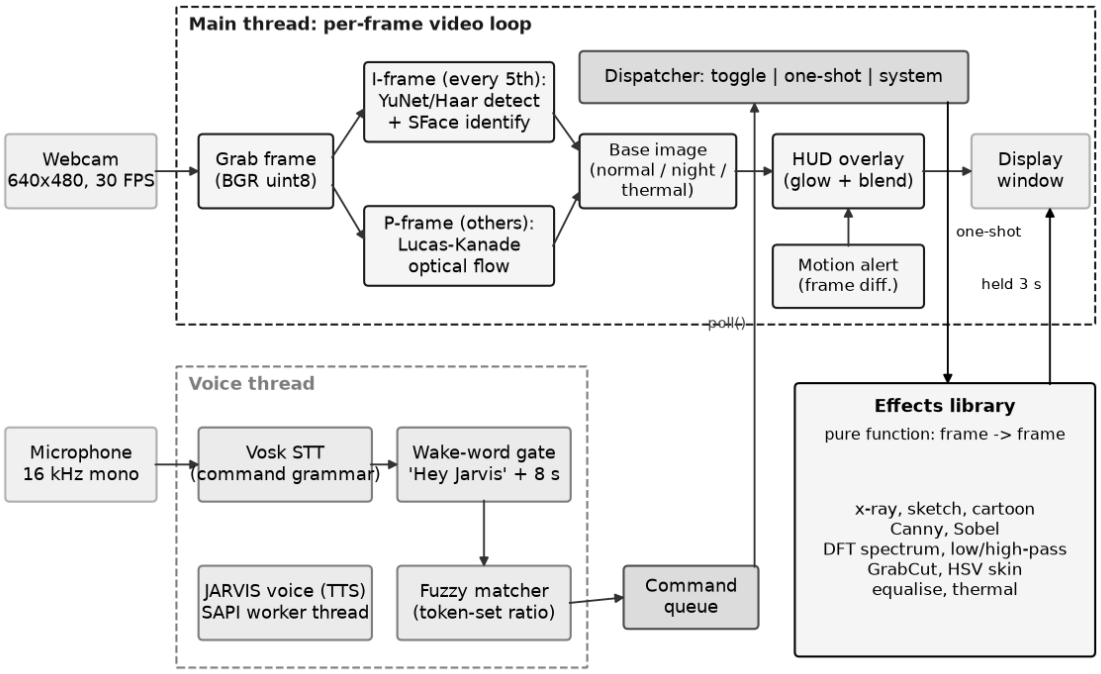


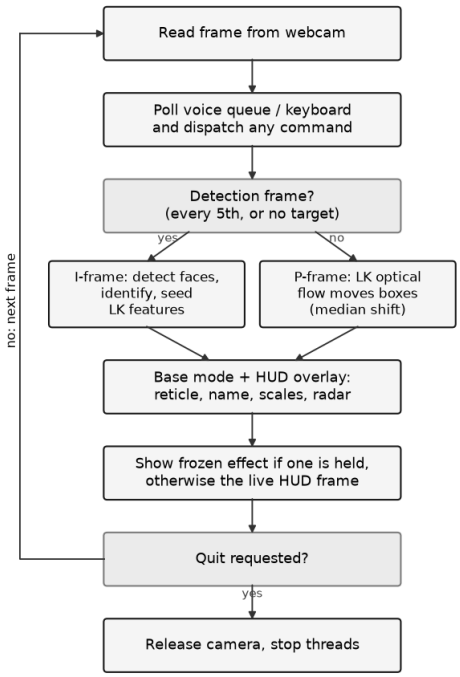
Figure 3. *System architecture: the video loop (main thread), the voice part (background thread), the command queue and the library of image-processing operations.*

Video loop (every frame)

1. **Read** a frame from the webcam.
2. **Run any command** waiting in the queue: a *toggle* switches a mode on or off, a *one-shot* applies an operation to the current frame and freezes the result on screen for 3 seconds, and a *system* command takes a snapshot, registers a face, or quits.
3. **Detect or track faces.** On every fifth frame (an I-frame), or when no face is being followed, detect all faces and recognise them. On the other frames (P-frames), estimate how each face moved using optical flow and move its box by that amount.
4. **Prepare the base image.** Optionally convert it to night vision or thermal, and optionally run frame differencing for the motion alert.
5. **Draw the HUD** on top: reticles, names, distance and direction, radar and status, blended with the frame.
6. **Show** the frozen result if one is being held, otherwise the live HUD.

Voice part (background thread)

Microphone audio goes to the Vosk speech recogniser, which only listens for the 146 known command phrases. A command is accepted only if it starts with “Jarvis” or comes within 8 seconds of the previous command. The words are matched to the closest command and placed in the queue. Replies are spoken by a separate thread so the video never waits, and anything heard while JARVIS is speaking is ignored so that he does not obey his own voice.



Pseudocode of the main loop

```
t <- 0
loop:
  frame <- camera.read();  t <- t + 1
  cmd <- voice_queue.poll() or keyboard()
  if cmd is TOGGLE:   switch the mode
  if cmd is ONESHOT:
      held <- operation[cmd](frame)  # 3 s
  gray <- grayscale(frame)
  if t mod 5 = 0 or no face is tracked:
      # I-frame
      boxes, points <- detect_faces(frame)
      names <- recognise(frame, points)
      start_tracking(gray, boxes)
  else:
      # P-frame: optical flow
      boxes <- track(gray)
  base <- night | thermal | frame
  out <- draw_hud(base, boxes, names)
  show(held if a result is held else out)
```

Figure 4. *Flowchart of the per-frame loop.*

6. Techniques / Algorithms Used

Table 2 lists the techniques by syllabus unit. The formulas below use the notation of the course notes: r = input pixel value, s = output pixel value, L = 256 gray levels for an 8-bit image.

Table 2. *Techniques used, by syllabus unit*

Syllabus unit	Technique	Where it is used
1. Image fundamentals	Image as a matrix f(x,y); sampling and quantisation (640×480, 8 bits); RGB image; Euclidean distance; file formats (JPEG, PNG, WebP)	Whole system; tracking error; snapshot file sizes
2. Point processing	Digital negative; contrast stretching; thresholding; log and power-law (gamma) transformations	X-ray; displaying results; motion alert; night vision
2. Neighbourhood processing	Mean, weighted mean (Gaussian), median and high-boost filters	Sketch; Canny; cartoon; sharpening
2. Histogram processing	Histogram equalisation; CLAHE (equalisation in small tiles)	"Enhance this"; x-ray; night vision; thermal
2. Frequency domain	Fourier spectrum; low-pass and high-pass filtering	"Smooth this", "sharpen this"
3. Image transforms	2-D DFT; DCT (inside JPEG)	Spectrum view; snapshot compression
4. Morphology	Erosion, dilation, opening, closing	Cleaning masks; motion regions; HUD glow
5. Edge detection	Sobel; Canny; Laplacian (as a sharpness measure)	"Find the edges"; Sobel view; analysis
6. Segmentation	Thresholding; component labelling (8-connectivity); foreground/background classification (GrabCut)	Skin and motion masks; counting regions; background removal
7. Colour processing	RGB, HSV and YUV (YCrCb) models; colour slicing; colour histogram processing; pseudo-colour	Skin; enhance; thermal; cartoon
8. Video	Temporal correlation; I- and P-frames; motion estimation	Face tracking; motion alert

Point processing and histogram equalisation

$$s = (L - 1) - r = 255 - r \quad \text{(digital negative, used by x-ray)}$$

$$s = (r - r_{min}) / (r_{max} - r_{min}) \times (L - 1) \quad \text{(contrast stretching, used to display results)}$$

$$s = c \log(1 + r) \quad \text{(log transformation, used to display the DFT)}$$

$$s = c\, r^{\gamma},\ \ r \text{ normalised to 0 to 1} \quad \text{(power law; } \gamma = 0.6 \text{ brightens dark areas in night vision)}$$

$$s = 255 \text{ if } r \geq T, \text{ otherwise } 0 \quad \text{(thresholding; T = 25 in the motion alert)}$$

$$s_k = \text{round}[\,(L - 1) \times CDF(r_k)\,] \quad \text{(histogram equalisation)}$$

As the notes show, equalising R, G and B separately changes the colours, so "enhance this" equalises only the brightness channel Y of the YCrCb (YUV) model.

Neighbourhood processing

A 3×3 or 5×5 mask slides over the image. The mean filter averages the neighbours, the weighted mean (Gaussian) filter gives the centre more weight, and the median filter takes the middle value, which removes salt-and-pepper noise but keeps edges. Sharpening uses the high-boost filter:

$$g = f + k\,(f - f_{blur}) \quad \text{(the "sharpen" command uses k = 2)}$$

Frequency domain and transforms

$$F(k,l) = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} f(m,n)\, e^{-j2\pi mk/M}\, e^{-j2\pi nl/N} \quad \text{(2-D DFT of an M×N image)}$$

The spectrum is shown after the log operator and rotated so that zero frequency is in the centre. Filtering multiplies the spectrum by a mask H and takes the inverse DFT (multiplication in the frequency domain equals convolution in the spatial domain). The low-pass mask is Gaussian, H =

$e^{-D^2/2D_0^2}$ with $D_0 = 24$, and the high-pass mask is 1 − H. JPEG compresses 8×8 blocks with the DCT:

$$X[k] = \alpha(k) \sum_{n=0}^{N-1} x[n] \cos[\ (2n + 1)\pi k / 2N\ ], \quad \alpha(0) = \sqrt{(1/N)}, \ \alpha(k) = \sqrt{(2/N)}$$

Morphology and edge detection

Erosion (A ⊖ B) keeps a pixel only where the structuring element B fits inside the object; dilation (A ⊕ B) sets it where B hits the object. Opening = (A ⊖ B) ⊕ B removes small noise, and Closing = (A ⊕ B) ⊖ B fills small holes. The Sobel masks give the derivatives Gx and Gy, with edge strength and direction

$$G = \sqrt{(Gx^2 + Gy^2)}, \quad \theta = \tan^{-1}(Gy / Gx)$$

Canny follows the four steps in the notes: 5×5 Gaussian noise reduction, Sobel gradient, non-maximum suppression, and hysteresis thresholding with minVal and maxVal. The system sets minVal = 0.67 × median and maxVal = 1.33 × median of the image, so one command works in bright and dark scenes.

Segmentation, colour and video

Skin is found by *colour slicing* in HSV: a pixel is kept if its hue is 0 to 20 (red to orange) and its saturation and value are high enough; hue changes much less with lighting than R, G and B do. Background removal uses GrabCut, which classifies every pixel as foreground or background using colour models of both, starting from a rectangle around the person. Regions in a mask are counted by component labelling with 8-connectivity.

Consecutive video frames are very similar (temporal correlation), so video compression codes I-frames fully and predicts P-frames using motion estimation. The system does the same: the full face detector runs on I-frames, and on P-frames only the motion is estimated. Instead of the block-matching searches in the syllabus (exhaustive, 2-D log and three-step search), it uses Lucas–Kanade optical flow, which estimates the motion of strong corner points inside each face box; the box moves by their median motion.

Faces are detected with YuNet, a small neural network, with the classical Haar cascade as fall-back. SFace turns each face into 128 numbers, and two faces are the same person if their cosine similarity is at least 0.40. Distance uses the pinhole camera relation d = f × W / w (W ≈ 15 cm face width, w = face width in pixels, f = focal length in pixels).

7. Implementation

The system was written in **Python** using **OpenCV** and **NumPy** for all image and video processing: about 2,900 lines of code in four parts (HUD, voice, image-processing operations and shared code), plus 82 automated tests. Table 3 lists the software and hardware.

Table 3. *Software, libraries and hardware*

Component	Version	Purpose
Python	3.14	Programming language
OpenCV	4.14	Camera input, all image and video processing, face detection and recognition models, display
NumPy	2.5	Matrix calculations and FFT
Vosk	0.3.45	Offline speech recognition (small English model)
sounddevice	0.5	Microphone input
Windows SAPI (comtypes)	1.4	JARVIS voice (male UK English voice “George”)
RapidFuzz	3.14	Matching spoken words to the closest command
pytest, Matplotlib	9.1, 3.11	Automated tests; charts for this report
Hardware	-	Laptop with Intel Core i5-1235U processor, 16 GB RAM, built-in 640×480 webcam and microphone, Windows 11 (no graphics card used)

Important implementation steps

1. **Common frame format first.** All parts agree that a frame is a 640×480×3 array of 8-bit values, so the HUD, voice and image-processing parts could be built and tested separately.
2. **Basic loop:** camera, keyboard commands and display, then face detection and the reticle, then the first operations, so a working program existed after every step.
3. **Tracking:** optical flow between detections, with the I/P frame type shown on the HUD.
4. **Image-processing library:** twelve operations, each a separate function, each tested automatically (same size, 8-bit, input unchanged, under one second).
5. **Voice:** command matching was tested with typed text first; then the microphone, wake word and spoken replies were added.
6. **HUD design:** all lines are drawn into masks, softened with a Gaussian blur for a glow, and blended with the frame in one step; parts that never change are drawn only once.
7. **Face recognition:** YuNet detection, SFace recognition, a local list of registered faces and the “remember me” command; replies use the person's name.
8. **Evaluation:** one script produces every figure and number in this report. It found a bug in voice matching, which was fixed (Section 8.8).

The program is started with “python main.py”. Every command also has a keyboard shortcut, so the system can be shown without a microphone.

8. Results and Discussion

Table 4 compares the results with the objectives of Section 2.2. The sections after it give the details. Unless stated otherwise, every result was produced from the team photo at 640×480.

Table 4. *Objectives and results*

Objective	Target	Result
1. Real-time HUD with detection, tracking and recognition	30 frames per second or more	35 frames per second (28.3 ms per frame) with four people in view
2. Hands-free voice control	95 % or more understood, no wrong commands	592/592 commands understood; 0/40 wrong commands from unrelated speech
3. Operations from all units	All 8 units, each under 1 s	All 8 units covered; slowest operation 157 ms (remove_bg)
4. Viewing modes	Night vision, thermal, motion alert	All three work as live modes
5. Measured results	Measured, repeatable	Speed, frame rate, tracking error, compression, recognition and voice accuracy (this section)

8.1 The heads-up display

Figure 5 shows the HUD in three states: searching when no face is visible, locking on (the rings shrink onto the nearest face), and locked. All four people in the photo are tracked; the nearest person is the main target and gets a data panel with distance and direction. After Gurmann registered his face, his name tag reads GURMANN while the others show UNKNOWN. On this photo the YuNet detector found all 4 faces, while the classical Haar cascade found only 1, which is why YuNet is used. Figure 6 shows the four live modes. The small dots on the faces are the corner points followed by optical flow on P-frames.

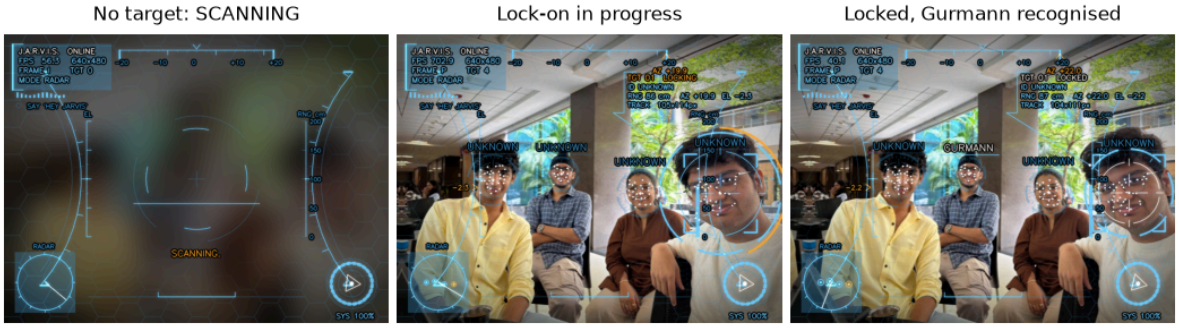


Figure 5. *HUD states: searching (no face), locking on, and locked with four people tracked and Gurmann recognised.*



Figure 6. *Live modes: normal HUD on a P-frame (tracked points visible), night vision, thermal and motion alert.*

8.2 Input and output of each operation

Figure 7 shows the input and the output of all twelve operations as they appear on screen. Figure 8 and Table 5 give the time each one takes on a 640×480 frame. Eleven of the twelve take 44 ms or less, so they could even run on every frame; background removal is the slowest at 157 ms, which is still well under the one-second target because the result is frozen on screen for 3 seconds. Background removal starts from the main (nearest) face, so it keeps only that person.

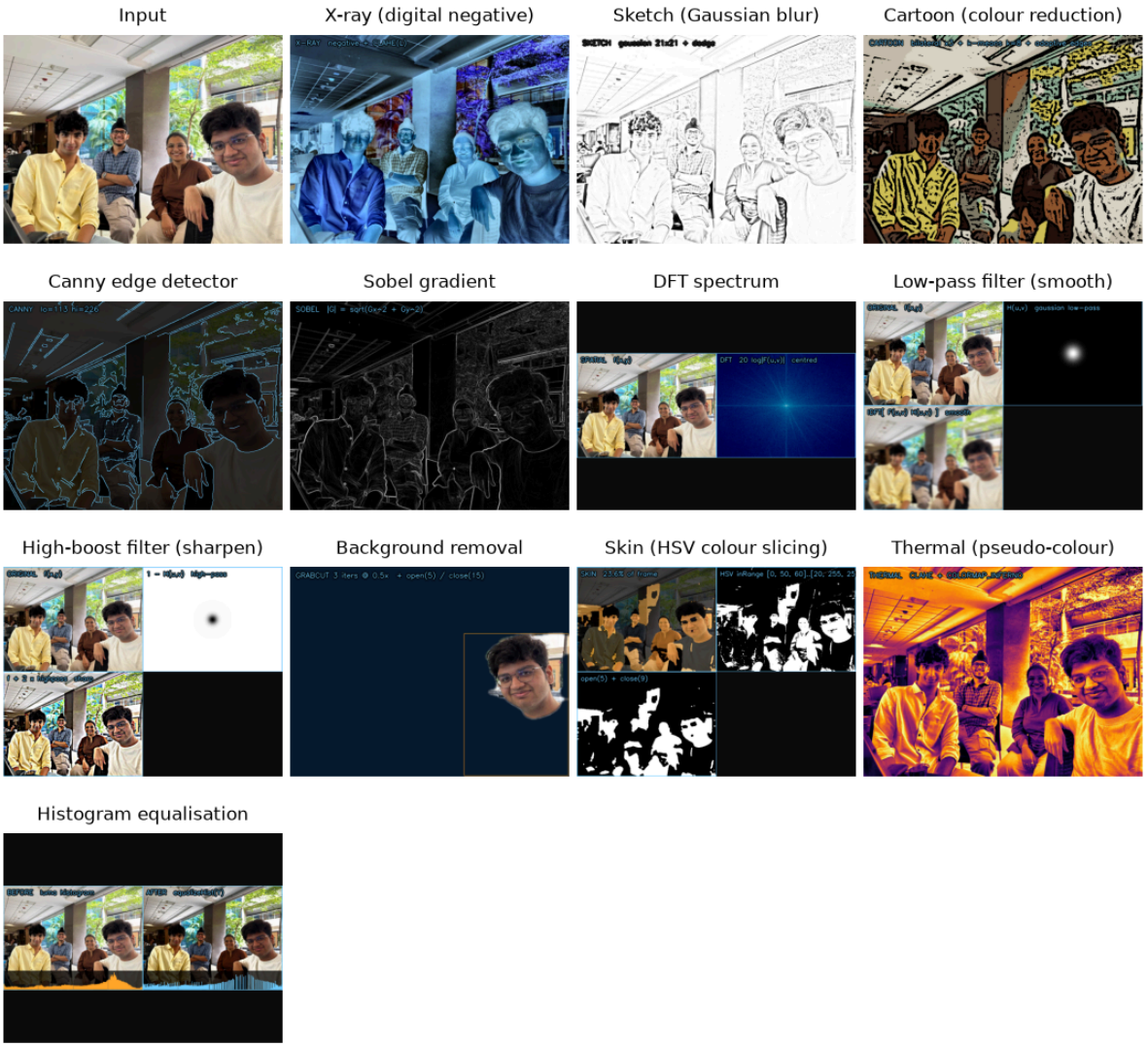


Figure 7. *The input frame and the output of every one-shot operation.*

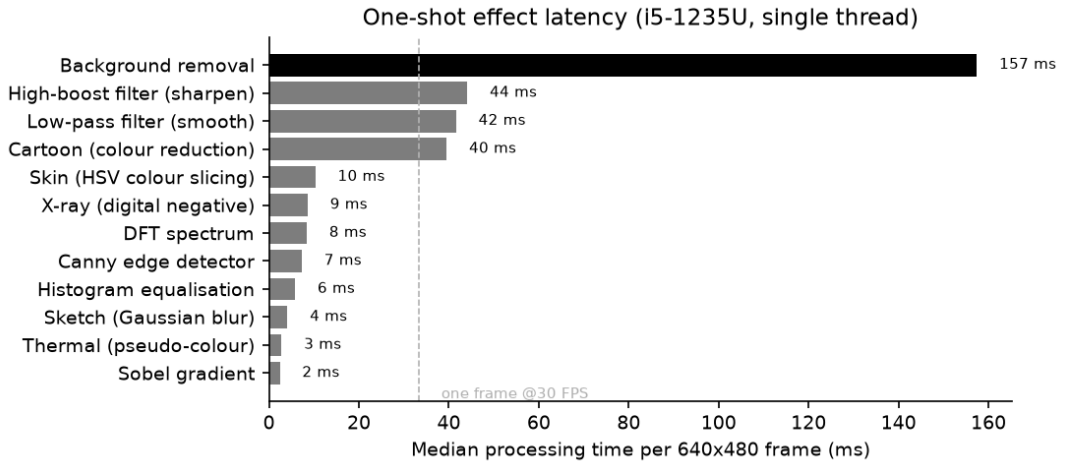


Figure 8. *Time per frame for each operation (dashed line: 33 ms, the time available for one frame at 30 frames per second).*

Table 5. *Time taken by each operation (640×480 frame, middle value of 15 runs)*

Operation	Time (ms)	Frames per second it could run at
sobel	2.4	417
thermal	2.8	357
sketch	4.1	244
enhance	5.8	172
edges	7.3	137
spectrum	8.4	119
xray	8.6	116
skin	10.3	97
cartoon	39.5	25
lowpass	41.6	24
highpass	44.1	23
remove_bg	157.4	6

8.3 Intermediate results

Edge detection

Figure 9 follows the Canny steps from the notes. The photo is bright, with a median gray level of 170, so the automatic thresholds were minVal = 113 and maxVal = 226; 5.3 % of pixels were marked as edges. Gx responds to vertical lines such as the pillar and the window frames, and Gy to horizontal lines such as the ceiling panels.

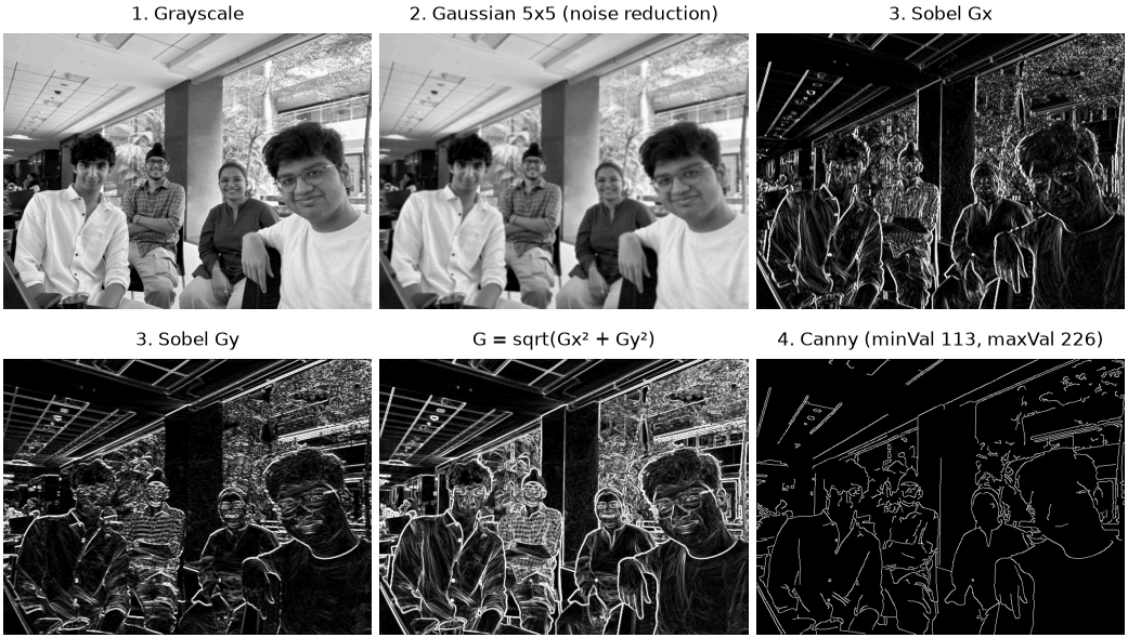


Figure 9. *Edge detection: grayscale, Gaussian noise reduction, Sobel Gx and Gy, gradient magnitude G, and the final Canny edges.*

Background removal and morphology

Figure 10 shows background removal step by step. The starting rectangle is placed around the nearest person's face and shoulders, and GrabCut classifies each pixel as foreground or

background. Opening (to remove small specks) followed by closing (to fill small holes) changed 0.34 % of the pixels, smoothing the edge of the mask; component labelling counted 1 foreground region after cleaning.

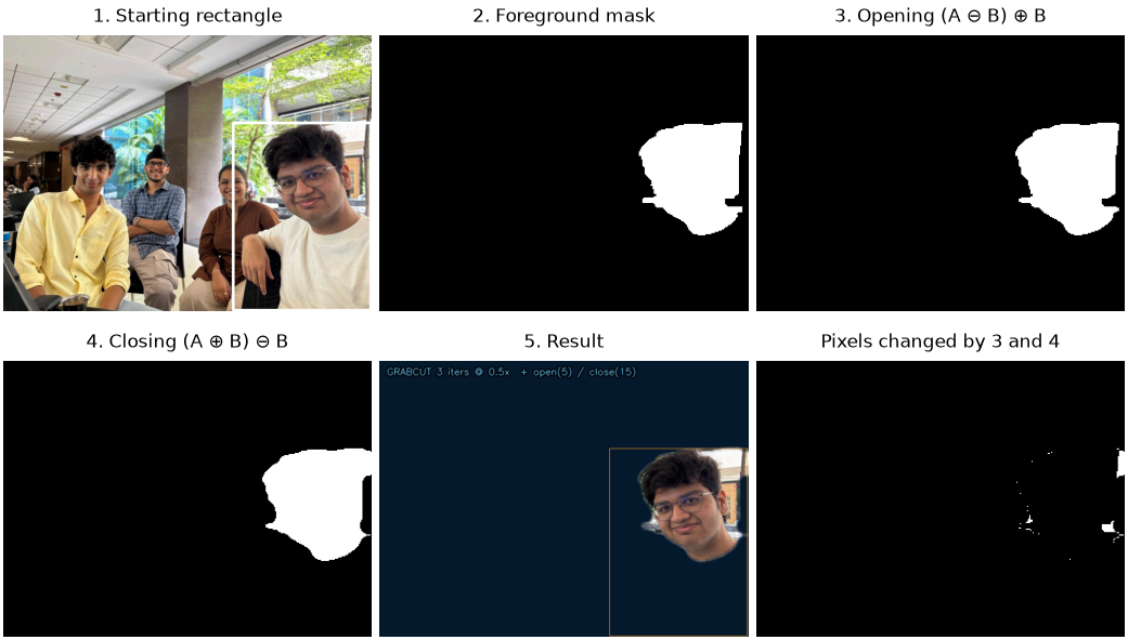


Figure 10. *Background removal: starting rectangle, foreground mask, opening, closing, result, and the pixels changed by opening and closing.*

Skin segmentation (colour slicing)

Colour slicing in HSV (Figure 11) produced 388 separate regions before cleaning and 31 after opening and closing, covering 23.6 % of the frame. The faces, hands and arms are found, but so are the wooden pillar and some warm-coloured background, because they have the same hue as skin. This is a known limit of segmenting by colour alone (Section 9).



Figure 11. *Skin segmentation: hue channel, colour slicing in HSV, mask after opening and closing, and the skin pixels kept.*

Fourier spectrum and frequency-domain filtering

In the centred log spectrum (Figure 12) the bright centre is the average brightness (zero frequency), and the bright lines through it come from the straight edges of the ceiling, pillar and windows. 92.1 % of the image's energy lies within 5 units of the centre and 97.1 % within 20, so most of the information is in the low frequencies. This is why the low-pass filter ($D_0 = 24$) removes fine detail but keeps the picture recognisable. To measure sharpness we used the variance of the Laplacian (a second-order derivative): it fell from 931 to 6.9 after low-pass filtering and rose to 5939 after high-boost sharpening.

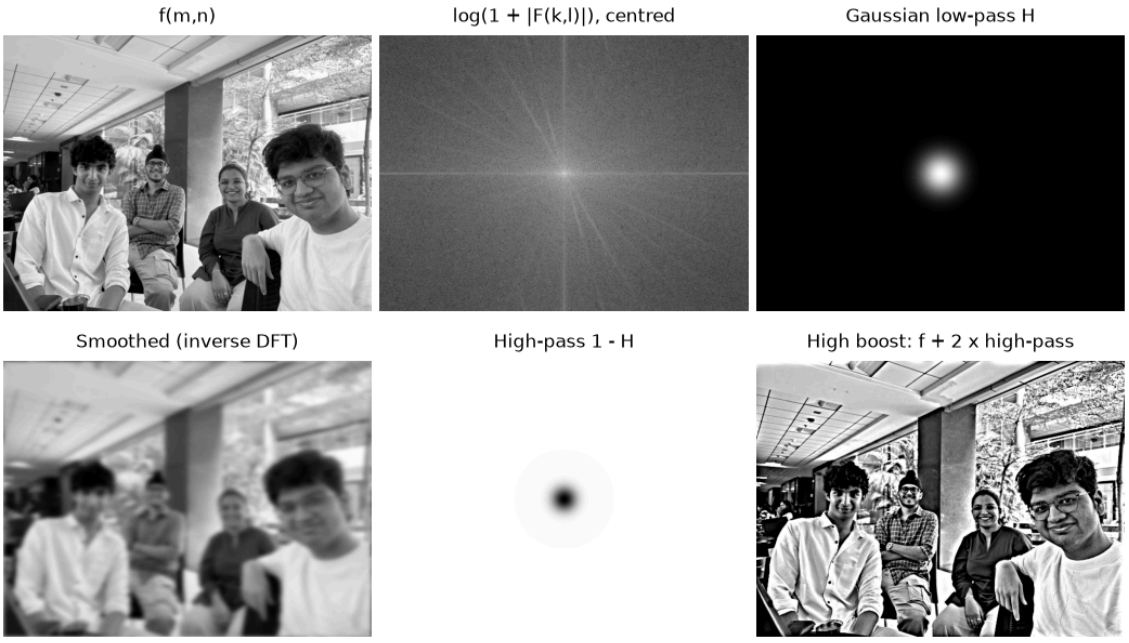


Figure 12. *Frequency domain: input, centred log spectrum, Gaussian low-pass mask and the smoothed result, high-pass mask and the high-boost (sharpened) result.*

Cartoon effect

The cartoon operation (Figure 13) first smooths the image with a bilateral filter, which smooths flat areas but keeps edges, then reduces the colours using k-means clustering: the half-size frame had 53,481 different colours, reduced to exactly 8. A thresholded edge mask adds the black outlines.



Figure 13. *Cartoon steps: bilateral filtering, reduction to 8 colours, edge mask by thresholding, final result.*

Histogram equalisation

Figure 14 and Table 6 compare the brightness channel before and after equalisation. The photo is fairly bright (mean 149.5), with a peak near 200 from the window light. After equalisation the CDF becomes almost a straight line, as expected from the formula in Section 6, the mean moves to the middle of the range (128.0) and the contrast (standard deviation) rises from 66.7 to 73.8. CLAHE equalises small tiles separately; it changes the image more gently and keeps local detail, so it is used in night vision and x-ray.

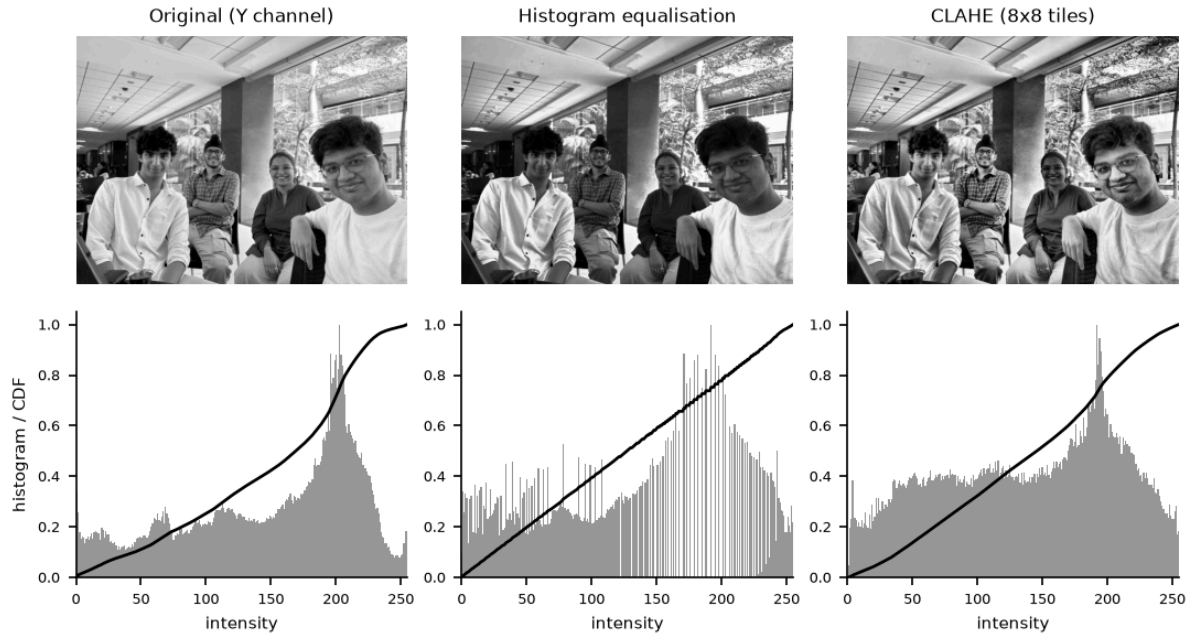


Figure 14. *Brightness channel and its histogram (bars) with the CDF (line): original, histogram equalisation, CLAHE.*

Table 6. *Brightness statistics before and after histogram processing*

Image	Mean	Standard deviation (contrast)	Range of 98 % of pixels
Original	149.5	66.7	3 to 250
Histogram equalisation	128.0	73.8	3 to 252
CLAHE (8×8 tiles)	138.1	67.8	5 to 251

Motion detection by frame differencing

Figure 15 shows the motion alert on two frames in which one region moved. The two blurred grayscale frames are subtracted, the difference is thresholded at T = 25, and dilation joins the pieces into one region. 9.2 % of the pixels changed, more than the 2 % alert limit, so the alert is raised.

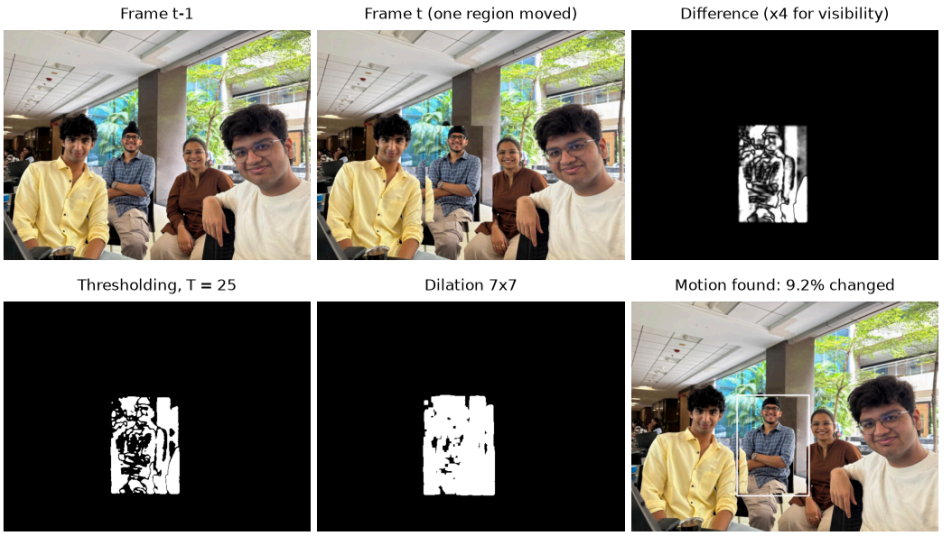


Figure 15. *Motion detection: previous and current frame, difference (×4 for visibility), thresholding, dilation, detected region.*

8.4 Image compression

Each snapshot reports how big it is in lossless and lossy formats. The uncompressed frame is 900 KB. Lossless PNG only reduced it to 563 KB (compression ratio 1.6 : 1), because a detailed photo has little repetition to remove. JPEG, which uses the DCT on 8×8 blocks and throws away small high-frequency coefficients, reached 9.5 : 1 at quality 90 with a PSNR of 37.0 dB, and 40.3 : 1 at quality 20 with 28.3 dB (Figure 16, Table 7). Here compression ratio = original size / compressed size, and PSNR = $10 \log_{10}(255^2 / \text{MSE})$ measures how close the decoded image is to the original. Figure 17 shows the 8×8 block pattern that appears at low quality.

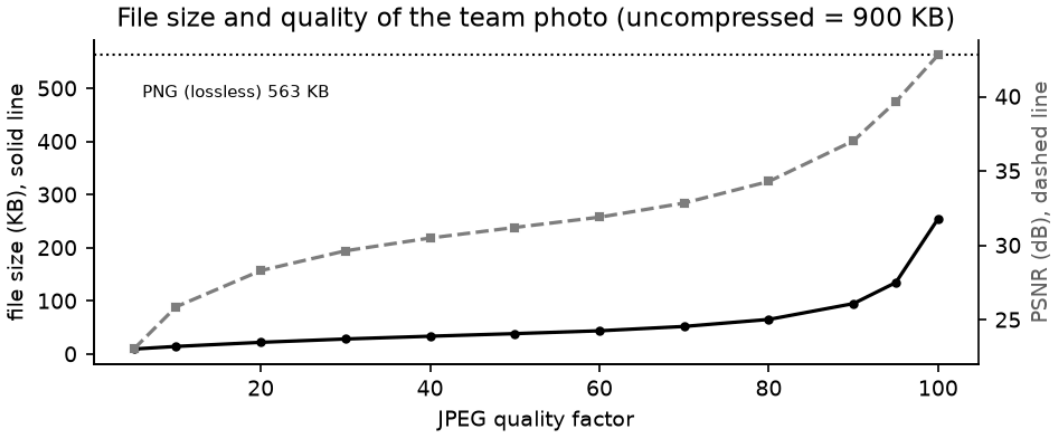


Figure 16. *JPEG file size (solid line) and PSNR (dashed line) against the quality setting; dotted line: PNG size.*

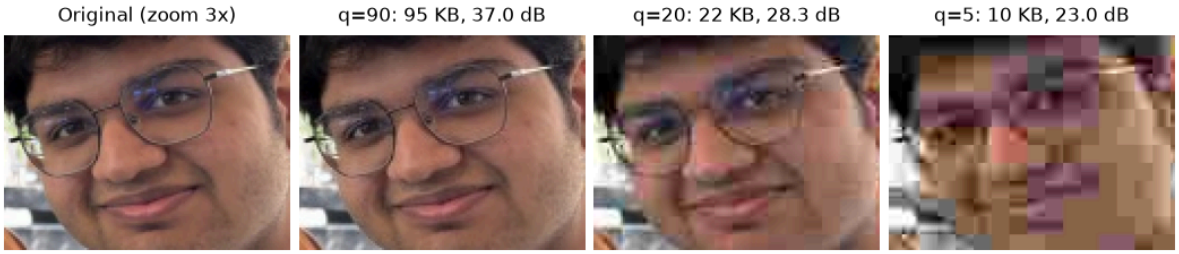


Figure 17. *Enlarged face at JPEG quality 90, 20 and 5: blocking appears as quality falls.*

Table 7. *Compression of the 640×480 team photo*

Format	Size (KB)	Compression ratio	Bits per pixel	PSNR (dB)
Uncompressed (3 × 8 bits)	900.0	1 : 1	24.00	-
PNG (lossless)	563.1	1.6 : 1	15.02	-
JPEG quality 95	134.8	6.7 : 1	3.60	39.66
JPEG quality 90	95.2	9.5 : 1	2.54	37.03
JPEG quality 70	52.2	17.2 : 1	1.39	32.85
JPEG quality 50	38.7	23.2 : 1	1.03	31.21
JPEG quality 20	22.3	40.3 : 1	0.59	28.31
JPEG quality 10	14.6	61.6 : 1	0.39	25.88
JPEG quality 5	9.7	93.1 : 1	0.26	23.04

8.5 Speed of the complete system

Figure 18 and Table 8 show the cost of each part and of the whole system with four people in view. Recognising a face costs about 11 ms, so recognising all four faces on every frame takes 83 ms per frame (12 frames per second), far below the camera's 30. With the I/P scheme, detection and recognition run only on every fifth frame and optical flow (about 2.2 ms per face) follows the faces in between. This brings the complete system down to 28.3 ms per frame, or 35 frames per second, above the camera rate.

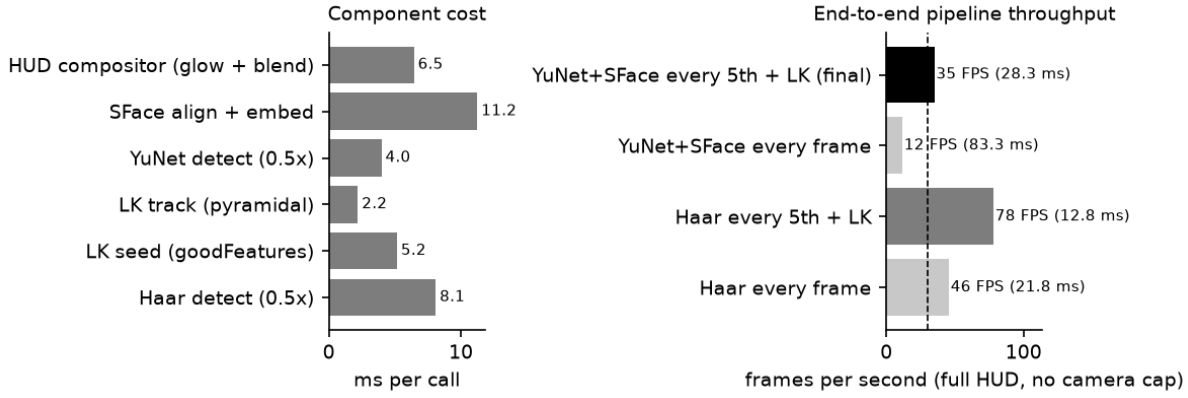


Figure 18. *Left: time taken by each part. Right: frames per second of the complete HUD with four strategies (dashed line: 30 frames per second).*

Table 8. *Time per frame of the complete HUD, four people in view (mean of 60 frames)*

Strategy	ms per frame	Frames per second
Haar every frame	21.8	46

Strategy	ms per frame	Frames per second
Haar every 5th + LK	12.8	78
YuNet+SFace every frame	83.3	12
YuNet+SFace every 5th + LK (final)	28.3	35

8.6 Tracking accuracy (motion estimation)

To measure tracking we used the synthetic video, in which the true position of the face is known in every frame, and measured the error as the Euclidean distance between the true and the estimated face centre (Figure 19, Table 9). Running the detector only on every fifth frame and keeping the old box in between is cheap, but the box falls behind the moving face: the mean error is 10.5 pixels and it reaches 26.8 pixels. Adding optical flow on the P-frames brings the mean error down to 4.0 pixels, the same as running the detector on every frame (4.1 pixels). The error on P-frames (4.02) and I-frames (4.12) is almost equal, so the prediction does not drift between detections. On its own, the fast YuNet detector is actually cheaper than tracking four faces (4.0 versus 10.4 ms per frame); the real saving of the I/P scheme is that the much slower recognition step runs only on I-frames (Section 8.5).

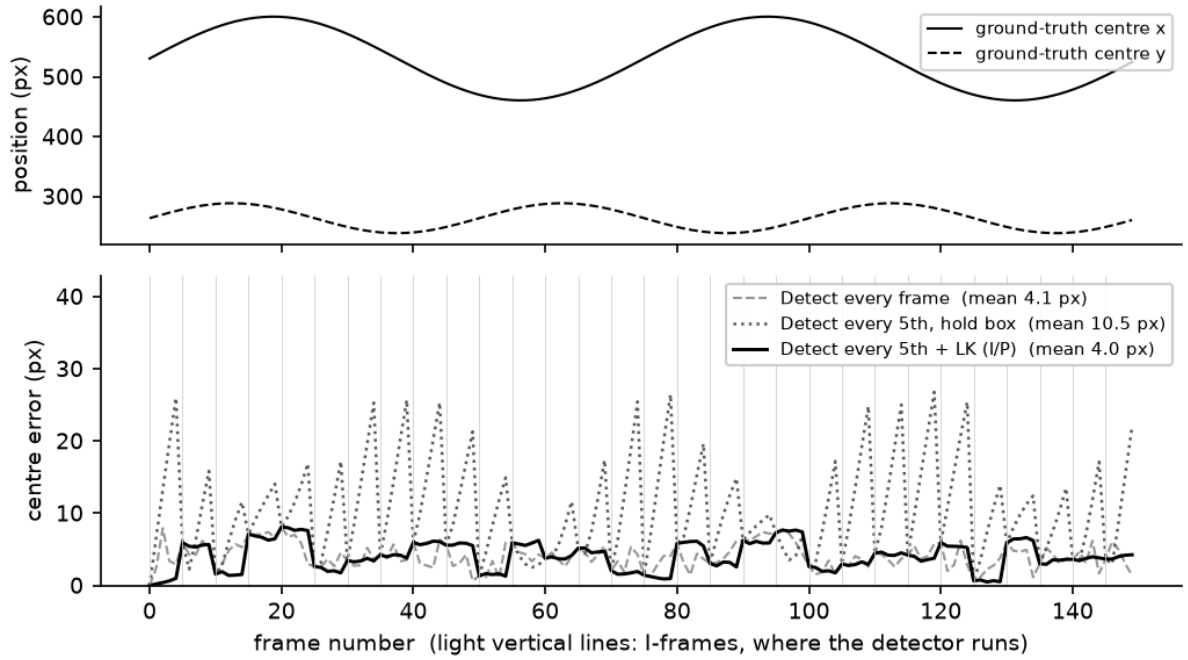


Figure 19. *True face position in the synthetic video (top) and the tracking error of the three strategies (bottom); light vertical lines mark I-frames.*

Table 9. *Tracking error on the 150-frame synthetic video*

Strategy	Mean error (px)	Error not exceeded in 95 % of frames (px)	Largest error (px)	Time per frame (ms)
Detect every frame	4.07	7.15	8.10	4.00
Detect every 5th, hold box	10.54	25.17	26.78	0.81
Detect every 5th + LK (I/P)	4.04	7.47	8.10	10.40

8.7 Face recognition

Gurmann's face was registered once, from a webcam frame. We then tested it in two ways (Figure 20, Table 10). First, the webcam frame was changed in ways a real camera changes it: darker, brighter, gamma, blur, noise, rotation, lower resolution, JPEG compression, mirror, grayscale and night vision. All of these were still recognised, with similarity between 0.65 and 0.98 (threshold 0.40). Second, we used the team photo, taken on a different day with a different camera. At full size (face 65x76 px) Gurmann was recognised with 0.54. After resizing the photo to the webcam size of 640×480, his face is only 32×41 pixels and the similarity dropped to 0.38, just below the threshold, so it was missed. This shows the effect of spatial resolution from Unit 1: a small, distant face has too few pixels to recognise reliably. The three teammates (0.06, 0.24, 0.20) and three other people from live sessions were all correctly rejected. In total 16 of 17 same-person tests and 6 of 6 different-person tests were correct. In the HUD figures Gurmann registered his face from the photo itself, the same way “remember me” works live.

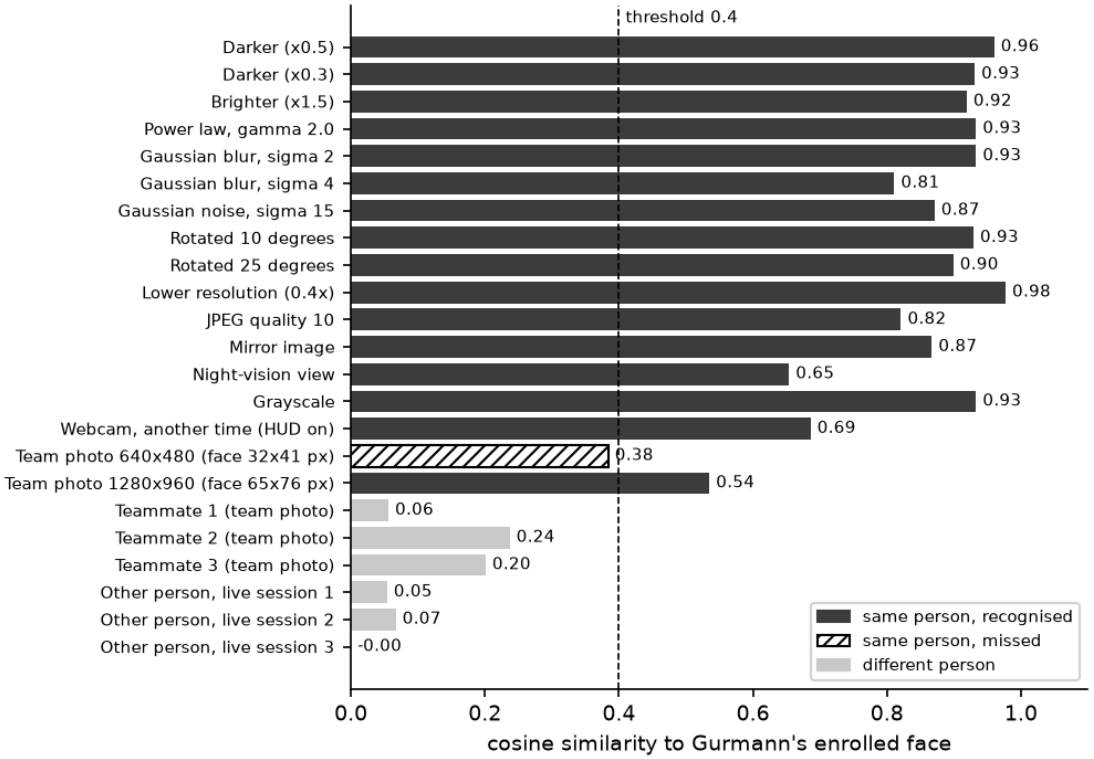


Figure 20. *Similarity to Gurmann's registered face. Dark bars: same person, recognised. Hatched bar: same person, missed. Light bars: other people. Dashed line: threshold 0.40.*

Table 10. *Face recognition tests*

Test image	Similarity	Decision
Darker (x0.5)	0.96	recognised (correct)
Darker (x0.3)	0.93	recognised (correct)
Brighter (x1.5)	0.92	recognised (correct)
Power law, gamma 2.0	0.93	recognised (correct)
Gaussian blur, sigma 2	0.93	recognised (correct)
Gaussian blur, sigma 4	0.81	recognised (correct)
Gaussian noise, sigma 15	0.87	recognised (correct)
Rotated 10 degrees	0.93	recognised (correct)
Rotated 25 degrees	0.90	recognised (correct)
Lower resolution (0.4x)	0.98	recognised (correct)

Test image	Similarity	Decision
JPEG quality 10	0.82	recognised (correct)
Mirror image	0.87	recognised (correct)
Night-vision view	0.65	recognised (correct)
Grayscale	0.93	recognised (correct)
Webcam, another time (HUD on)	0.69	recognised (correct)
Team photo 640x480 (face 32x41 px)	0.38	missed
Team photo 1280x960 (face 65x76 px)	0.54	recognised (correct)
Teammate 1 (team photo)	0.06	rejected (correct)
Teammate 2 (team photo)	0.24	rejected (correct)
Teammate 3 (team photo)	0.20	rejected (correct)
Other person, live session 1	0.05	rejected (correct)
Other person, live session 2	0.07	rejected (correct)
Other person, live session 3	-0.00	rejected (correct)

8.8 Voice commands

Every one of the 148 command phrases, said after “hey jarvis”, was produced by four different computer voices (three female, one male) and passed through the full voice system (Table 11). All 592 were understood and the wake word was heard every time. Recognising one second of speech took about 22 ms, so the main delay is the short pause (about 0.5 s) the recogniser waits for to know the speaker has finished.

The test also found a real bug. Ten unrelated sentences were said after the wake word for each voice, and at first **8 of 40** were wrongly turned into commands. For example, “good morning everyone” was heard as “[unknown] everyone” and matched to *forget everyone*, which deletes all registered faces. The fix was to accept a partly understood sentence only if the understood words form a complete command. After the fix there were **0 wrong commands** and all 592 real commands were still understood.

Table 11. *Voice command test*

Voice	Commands understood	Wake word heard	Wrong commands, before fix	Wrong commands, after fix
Hazel	148 / 148	100 %	3 / 10	0 / 10
Zira	148 / 148	100 %	1 / 10	0 / 10
George	148 / 148	100 %	2 / 10	0 / 10
Susan	148 / 148	100 %	2 / 10	0 / 10

Computer voices are clearer than a real person in a noisy classroom, so these results are a best case. In live use, the main problem we found was an early version that ignored a second command unless “Jarvis” was said again; this led to the 8-second follow-up window and the LISTENING sign on the HUD.

8.9 Use in live sessions

During development and practice the system was used on 2 days and saved 56 results (Table 12). X-ray and edge detection were used most often. All of them were triggered by voice or keyboard while the HUD kept running at the camera frame rate.

Table 12. *Results saved during live sessions, by command*

Command	xray	edges	snapshot	cartoon	remove_bg	lowpass	spectrum	highpass	skin	sobel	enhance
Count	13	12	7	5	5	4	3	2	2	2	1

9. Challenges / Limitations

Problems we faced

- **Library change:** OpenCV version 5 removed the Haar face detector, so the project uses OpenCV 4 and later added the YuNet detector.
- **Voice output:** the first text-to-speech library spoke only the first reply and had only female voices; the system now uses the Windows speech engine directly with a male voice.
- **Network:** the campus network blocked the server for the speech model, so the program falls back to typed commands until the model is installed by hand.
- **Wake word:** the first version ignored any command that did not start with “Jarvis”; an 8-second follow-up window and an on-screen LISTENING sign fixed this.
- **Wrong commands:** partly understood sentences were matched to wrong commands (Section 8.8); fixed.

Difficult cases and failures

- **Small or turned faces:** the Haar detector found only 1 of the 4 faces in the team photo, and recognition missed Gurmann when his face was only 32×41 pixels. Fast head movement can also lose a face until the next I-frame.
- **Colour segmentation:** skin detection by colour slicing also picks up wood, beige walls and warm light, and one fixed hue range does not suit every skin tone or light source.
- **Distance estimate** assumes every face is 15 cm wide, so it is only approximate for children or turned faces.
- **Speech** becomes less reliable with background noise and strong accents; our tests used computer voices, not real speakers.

Computational and data limitations

- Everything runs on the processor only; with many faces in view the frame rate falls because recognition is the slowest step.
- Recognition was tested with one registered person and a small number of other faces, which shows how robust it is but is not enough to give an error rate for many people.
- The tracking test moves a still photo sideways and up and down; real motion also includes turning, moving closer and being hidden behind objects.
- “Thermal” and “x-ray” are only visual effects based on brightness, not real measurements.

10. Conclusion

What was implemented. A touchless vision assistant: a real-time heads-up display on a laptop webcam that detects, tracks and recognises faces, shows distance, direction and name, offers night-vision, thermal and motion-alert modes, and applies twelve image-processing operations on spoken command, with a voice assistant that answers users by name. Everything runs offline on an ordinary laptop.

Techniques used. Point processing (digital negative, contrast stretching, thresholding, log and power-law transformations), histogram equalisation, mean, Gaussian, median and high-boost filters, the Fourier spectrum and frequency-domain filtering, the DFT and the DCT inside JPEG, erosion, dilation, opening and closing, Sobel and Canny edge detection, thresholding, component labelling and foreground/background segmentation, the RGB, HSV and YUV colour models with colour slicing and pseudo-colour, and I/P frames with motion estimation.

What was achieved. Every unit of the syllabus is shown by a working live feature. With four people in view the complete system runs at 35 frames per second, because the slow recognition step runs only on I-frames (it would be 12 frames per second otherwise). Optical-flow tracking reduced the tracking error from 10.5 to 4.0 pixels compared with simply holding the last detection. Voice commands were understood in 592 of 592 tests with no wrong commands after a bug found by our own testing was fixed, and recognition was correct in 22 of 23 tests.

What we learned. Operations that are simple on one image behave differently in a live system, where every millisecond is shared between many steps. The I-frame and P-frame idea from video compression turned out to be the most useful speed-up in the whole project. Choosing the right colour model or threshold (HSV for skin, only the Y channel for equalisation, thresholds based on the median for Canny) mattered as much as choosing the algorithm. Spatial resolution also matters: the same face recognised at full size was missed when it was only 32 pixels wide. Finally, measuring the system, instead of only looking at the output, showed us both its strengths and a real bug that normal testing had missed.

Future work. An infrared or depth camera for real temperature and distance, a graphics card so that background removal can run live, testing recognition on more people, and testing the voice control with real speakers in a noisy room.

References

[1] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. Pearson, 2018.

[2] W. Burger and M. J. Burge, *Digital Image Processing: An Algorithmic Introduction*, 2nd ed. Springer, 2022.

[3] A. McAndrew, *A Computational Introduction to Digital Image Processing*, 2nd ed. CRC Press, 2021.

[4] R. G. Mishra, *Image and Video Processing: course lecture notes* (Units 1 to 5), SVKM's NMIMS MPSTME, 2026.

[5] N. Ahmed, T. Natarajan, and K. R. Rao, "Discrete cosine transform," *IEEE Trans. Computers*, vol. C-23, no. 1, pp. 90–93, 1974.

[6] G. K. Wallace, "The JPEG still picture compression standard," *IEEE Trans. Consumer Electronics*, vol. 38, no. 1, 1992.

[7] J. Canny, "A computational approach to edge detection," *IEEE Trans. Pattern Analysis and Machine Intelligence*, vol. 8, no. 6, pp. 679–698, 1986.

[8] B. D. Lucas and T. Kanade, "An iterative image registration technique with an application to stereo vision," in *Proc. IJCAI*, 1981, pp. 674–679.

[9] J. Shi and C. Tomasi, "Good features to track," in *Proc. IEEE CVPR*, 1994, pp. 593–600.

[10] C. Rother, V. Kolmogorov, and A. Blake, "GrabCut: Interactive foreground extraction using iterated graph cuts," *ACM Trans. Graphics*, vol. 23, no. 3, pp. 309–314, 2004.

[11] P. Viola and M. Jones, "Rapid object detection using a boosted cascade of simple features," in *Proc. IEEE CVPR*, 2001.

[12] W. Wu, H. Peng, and S. Yu, "YuNet: A tiny millisecond-level face detector," *Machine Intelligence Research*, vol. 20, no. 5, pp. 656–665, 2023.

[13] Y. Zhong, W. Deng, J. Hu, D. Zhao, X. Li, and D. Wen, "SFace: Sigmoid-constrained hypersphere loss for robust face recognition," *IEEE Trans. Image Processing*, vol. 30, pp. 2587–2598, 2021.

[14] Alpha Cephei, "Vosk offline speech recognition." [Online]. Available: https://alphacephei.com/vosk/

[15] G. Bradski, "The OpenCV Library," *Dr. Dobb's Journal of Software Tools*, 2000.